

Section 11.1 of 10: Ingestion, Context Extraction & Vector Search Optimization

Executive Overview

RAG systems form the backbone of enterprise AI applications, transforming unstructured data into searchable, context-rich vectors. This section covers the end-to-end pipeline: document ingestion, embedding generation, vector indexing, and retrieval optimization. Interviewers probe understanding of similarity metrics, index selection trade-offs, and the latency/accuracy curve in production retrieval systems.

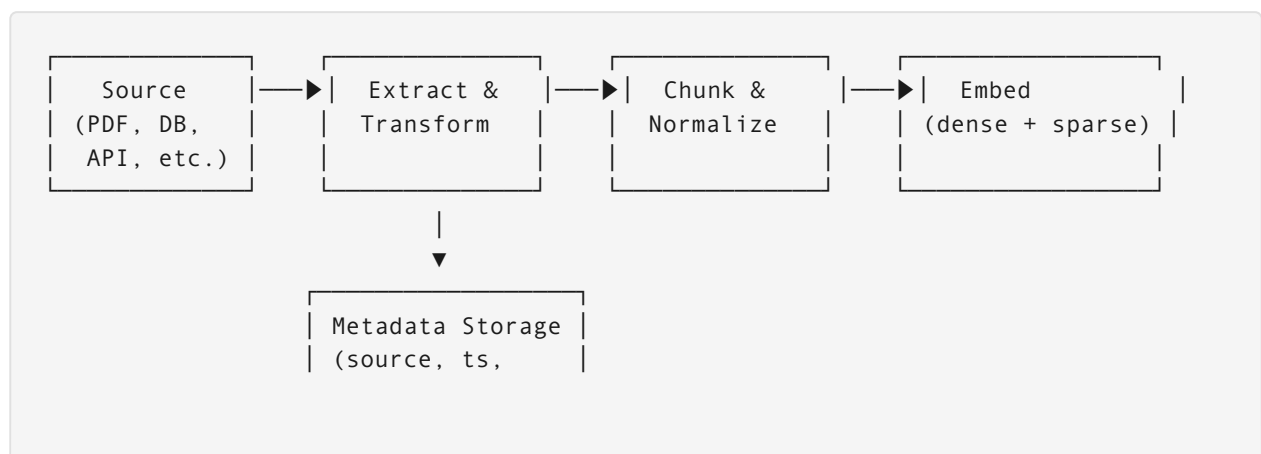
Math-heavy alert: Similarity metrics involve vector arithmetic in high-dimensional spaces. Extra weight is given to worked numeric examples and proof-by-contradiction style explanations for why certain metrics fail or degrade.

11.1.1 Multi-Modal Enterprise Ingestion Pipelines

What It Is

Enterprise data exists across formats: PDFs, images, videos, databases, APIs. An ingestion pipeline normalizes this into embeddings while preserving metadata and provenance.

Pipeline Architecture



author, page)

Document Processing Flow: 1. **Extraction:** Apache Tika (PDF/Word), OCR (images), table parsers 2. **Sanitization:** PII redaction, format normalization, character encoding 3. **Chunking:** Semantic splitting using sentence boundaries, paragraph markers 4. **Metadata association:** Source, timestamp, author, document ID, page number

Chunking Strategy Comparison

Stage	Latency	Accuracy	Complexity	When to Use
Naive chunking (fixed tokens)	Low	Poor — splits context mid-sentence	Low	Prototyping only
Semantic chunking (sentence/paragraph boundaries)	High	Good — respects natural boundaries	High	Most production systems
Parent-document tracking (small child chunks, large parent retrieved)	Medium	Excellent — precision retrieval, rich context	Very High	Legal, medical, compliance docs

Key insight on chunking: Naive chunking at 512 tokens can split “The defendant was found NOT guilty” across two chunks — embedding the first half alone gives a semantically opposite signal. Semantic chunking prevents this by respecting sentence boundaries.

Failure Modes & Mitigation

Failure	Root Cause	Mitigation
OCR errors on scanned docs	Image quality, fonts	Confidence threshold + fallback to image embedding
PII leakage in embeddings	No pre-processing	Pre-embedding redaction, differential privacy, access controls

Failure	Root Cause	Mitigation
Back-pressure	High ingestion rate	Async queue (Kafka/SQS), backpressure signaling
Incremental ingestion drift	Full re-index not triggered	Change-data-capture (CDC) triggers, document fingerprinting

11.1.2 Spatial Coordinate Mathematics — Similarity Metrics

Give extra weight to worked numeric examples here. These metrics are frequently tested with on-the-spot arithmetic.

Core Distance Metrics

1. Euclidean Distance (L2)

$$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

Worked example: Two embeddings in 3D space: - $x = [1, 2, 3]$, $y = [4, 6, 3]$ - $d = \sqrt{((4-1)^2 + (6-2)^2 + (3-3)^2)} = \sqrt{(9 + 16 + 0)} = \sqrt{25} = 5$

Problem: Euclidean is sensitive to vector *magnitude*, not just direction. A document repeated twice produces a vector $\sim 2\times$ as long; Euclidean incorrectly treats it as far from the original.

2. Cosine Similarity

$$\text{sim}(x, y) = (x \cdot y) / (||x|| \times ||y||)$$

Worked example: $x = [1, 0, 1]$, $y = [2, 0, 2]$ - Dot product: $1 \times 2 + 0 \times 0 + 1 \times 2 = 4$ - $||x|| = \sqrt{2}$, $||y|| = \sqrt{8} = 2\sqrt{2}$ - $\text{sim} = 4 / (\sqrt{2} \times 2\sqrt{2}) = 4/4 = \mathbf{1.0}$ (identical direction, different magnitude)

Proof by contradiction why this matters: Suppose you use L2 for semantic search. A 2-page document embedding will have magnitude $\sim \sqrt{2}\times$ a 1-page document with the same content. L2 would rank them far apart despite identical meaning. Cosine cancels magnitude, correctly ranking them as near-identical.

3. Dot Product

$$x \cdot y = \sum (x_i \times y_i)$$

Equivalent to cosine if vectors are unit-normalized. Preferred in ANN indices because it avoids the division step.

Curse of Dimensionality — Worked Explanation

In 2D, two random unit vectors have expected angular distance of $\sim 90^\circ$. In 1024D:

$$E[\text{dist}(x,y)] \rightarrow \mu \pm \sigma \text{ where } \sigma/\mu \rightarrow 0 \text{ as } d \rightarrow \infty$$

What this means practically: In 1024-dimensional space, ALL pairs of random vectors converge to roughly the same distance (~ 0.7071 for cosine). The signal-to-noise ratio of your similarity scores collapses.

Mitigation: Dimensionality reduction (PCA, random projection) or use asymmetric quantization.

Metric Selection Guide

Metric	Best For	Avoid When	Compute Cost
Euclidean	Image embeddings, spatial data	Variable-length text docs	High ($\sqrt{}$ operation)
Cosine	Text embeddings, NLP tasks	Binary vectors	Medium (division)
Dot product	Normalized vectors, fast ANN	Un-normalized embeddings	Low

11.1.3 Distributed Vector Database Architecture

Index Types Deep Dive

HNSW (Hierarchical Navigable Small World)

Layer 2 (long-range links, few nodes): 
 Layer 1 (medium links): 
 Layer 0 (all nodes, fine-grained): 

- Greedy search starts at top layer, descends toward query

- **M** = connections per node (16–64 typical); higher M = better recall, more memory
- **efSearch** = beam width during query; higher = better recall, higher latency

Worked numeric trade-off: - M=16, efSearch=16 → ~85% recall, 2ms latency - M=32, efSearch=64 → ~97% recall, 8ms latency - Choose based on your recall SLA

IVF (Inverted File Index)

k-means → C clusters
Query → find closest nprobe clusters → search only those

- **nprobe=1**: Fastest, lowest recall (only searches nearest cluster centroid)
- **nprobe=100**: Near-exhaustive, high recall, slow
- Rule of thumb: $nprobe = \sqrt{C}$ for balanced trade-off

Product Quantization (PQ)

Compresses 768-float vectors (1536 bytes) into 8-byte codes using a learned codebook. - IVF-PQ combines coarse IVF filtering with PQ fine-grained search - Loss: ~5–15% recall drop vs. flat index; gain: 100× memory reduction

Index Selection Guide

Index Type	Recall	Latency	Memory	Dataset Size
HNSW	95–99%	1–5ms	High (full vectors in RAM)	<100M vectors
IVF-Flat	95–99%	5–20ms	Very High	10M–1B vectors
IVF-PQ	85–95%	2–10ms	Low (compressed)	100M–10B vectors
ScaNN	90–98%	1–3ms	Medium	10M–1B vectors

Failure Modes

Failure	Root Cause	Mitigation
Hot vectors causing load imbalance	Popular embeddings hit same shard	Shuffle sharding, query-layer load shedding
	Soft deletes orphan graph nodes	

Failure	Root Cause	Mitigation
HNSW graph fragmentation under deletes		Scheduled re-index, soft deletes + background cleanup
Index staleness	New docs not indexed	CDC-triggered incremental index updates

11.1.4 Retrieval-Augmented Generation (RAG) Architecture

Pipeline Evolution

Naive RAG:

Query → Embed → Vector DB → Top-K results → Concatenate → LLM

Problem: Lexical mismatches (“myocardial infarction” vs “heart attack”) cause missed results.

Production RAG:

Query → Rewriter → BM25 (sparse) → RRF Fusion → Reranker → Context → LLM
 → Dense Embeddings

Hybrid Retrieval Components

Component	What It Does	When to Use
BM25 (sparse)	Keyword/lexical match	Exact terminology required (legal, medical)
Dense embeddings	Semantic similarity	Paraphrase-heavy domains (customer support)
RRF Fusion	Rank-based score merging	Always — no hyperparameter tuning needed
Cross-encoder reranker	Deep query-passage scoring	When top accuracy matters, latency allows
ColBERT (late interaction)	Per-token interaction scoring	Balance of accuracy and speed

Reranker Comparison

Approach	Accuracy	Latency	How It Works
Bi-encoder	Medium	Fast (pre-computed passage vectors)	Query and passage encoded separately
Cross-encoder	High	Slow (full attention over query+passage)	Joint encoding captures fine-grained relevance
ColBERT	High	Medium	Token-level late interaction

Proof by example why cross-encoder beats bi-encoder: For query “bank loan interest rate,” a bi-encoder passage vector for “bank” encodes both financial institution and river bank senses. Cross-encoder sees the full query context and attends to “loan” to disambiguate. Result: cross-encoder precision is typically 5–15% higher.

Interview Problems

Problem 1: RAG Latency Optimization (p99 < 200ms)

Scenario: Legal document search, 200ms p99 SLA required.

Latency budget breakdown:

Stage	Target	Technique
Query embedding	10ms	GPU batch inference, cache frequent queries
Vector search	25ms	HNSW efSearch=50, metadata pre-filter
Reranking	60ms	ColBERT (2× faster than cross-encoder), max 512 tokens
LLM generation	80ms	Prompt caching for static prefix, 8K context max
Network overhead	25ms	Co-locate components, keep in same AZ
Total	200ms	

Accuracy safeguards: RAGAS metrics (faithfulness, context precision, recall); confidence threshold triggers “insufficient context” response.

Problem 2: Chunking Strategy for Multi-Format Documents

Scenario: Enterprise knowledge base with PDFs, Word docs, and database records. Documents range from 1-page memos to 500-page manuals.

Solution:

```
Short docs (<5 pages):      Single chunk + full metadata
Medium docs (5-50 pages):   Semantic chunking, 512 tokens, 10% overlap
Long docs (50+ pages):      Parent-document retrieval
    - Child chunks: 256 tokens (for precise retrieval)
    - Parent chunks: 2048 tokens (for rich context delivery to LLM)
Database records:           Schema-aware chunking per entity
```

Key design choice: Parent-document retrieval solves the “precision vs. context” tension — retrieve with small chunks (high precision), but deliver large parent chunks to the LLM (full context).

Problem 3: Handling Embedding Model Updates

Scenario: You upgrade your embedding model (e.g., text-embedding-ada-002 → text-embedding-3-large). Vectors are incompatible. You have 50M documents. How do you migrate?

Solution framework:

1. **Shadow indexing:** Build new index in parallel; serve from old index
2. **Dual-read phase:** Query both indexes, merge results via RRF
3. **Background re-embedding:** Process documents in batches (rate-limit to avoid GPU saturation)
4. **Cutover:** Once >95% documents re-embedded, switch primary index
5. **Cleanup:** Deprecate old index after 1 week monitoring

Why not cutover immediately? During re-embedding, new docs would go to new index but existing docs are in old index — hybrid search covers both until migration is complete.